

GitHub API Data Extraction Methodology

1. Purpose and Scope

The purpose of this data extraction exercise was to systematically collect grey literature from GitHub repositories that intersect with open-source software and artificial intelligence-related technologies, in support of DSIT's broader research on AI security, open-source ecosystems, and associated risks.

GitHub was selected as a primary source due to its role as the largest global repository of open-source software development, hosting both experimental and production-grade AI tooling, libraries, and infrastructure projects. The scope of the extraction focused on repositories that demonstrate meaningful relevance to AI development, deployment, security, governance, or misuse, rather than general software repositories.

2. Data Source

Data were collected exclusively via the GitHub REST API, using authenticated requests to ensure stability, reproducibility, and compliance with GitHub's usage policies. The API was used to retrieve structured metadata for public repositories, including repository names, descriptions, URLs, primary programming language, star counts, and creation and update timestamps. The extraction was limited to publicly available repositories, ensuring transparency and adherence to ethical standards. No private repositories or personal user data were accessed.

3. Search Strategy and Query Design

3.1 Conceptual Framework

The search strategy was designed around two conceptual term groups:

- Open-source terminology (e.g., open-source software, OSS, FOSS)
- Artificial intelligence terminology (e.g., machine learning, large language models, neural networks, AI security)

The analytical objective was to capture repositories that reflect the intersection of these two domains, rather than either domain in isolation.

3.2 API Constraints and Query Decomposition

The GitHub API does not support complex Boolean logic such as nested OR statements, parentheses, or wildcard-heavy expressions within a single query. To preserve the intended conceptual logic while remaining API-compliant, the original Boolean expression was decomposed into pairwise query combinations.

Each open-source-related term was paired individually with each AI-related term, resulting in a comprehensive set of discrete, GitHub-compatible search queries. These queries were executed sequentially and programmatically, ensuring full coverage of the conceptual search space.

This approach enabled the project to maintain methodological rigor while operating within the platform's limitations.

4. Data Extraction Process

4.1 Automated Collection

For each generated query, the GitHub Search API was used to retrieve repository-level results. The extraction pipeline implemented:

- Safe pagination to retrieve results across multiple pages
- Rate-limit monitoring and error handling
- Automated pauses and cooldowns following HTTP 403 responses
- Structured storage of retrieved metadata into tabular format

At the conclusion of the API-level collection, the dataset contained 71,395 raw repository records, representing the unfiltered output of the search strategy.

4.2 Data Consolidation

Because the same repository could appear in multiple query results, the raw output intentionally prioritised recall over precision. As a result, the initial dataset included duplicate entries across queries. These were retained at the extraction stage to avoid premature exclusion of potentially relevant repositories.

5. Identified Challenges and Mitigation Strategies

5.1 Unstructured and Dispersed Grey Literature

GitHub repositories are highly heterogeneous in structure, documentation quality, and intent. Repository descriptions vary significantly in depth and clarity, and many projects lack consistent metadata.

Mitigation:

A hybrid approach was adopted, combining automated extraction with downstream manual validation to ensure relevance and interpretability.

5.2 GitHub API Rate Limitations

The GitHub API enforces strict rate limits, including temporary access restrictions when limits are exceeded.

Mitigation:

The pipeline implemented safe pagination, request throttling, and enforced cooldown periods (including 60-second pauses following 403 responses) to ensure uninterrupted data collection and compliance with GitHub policies.

5.3 High Volume of Retrieved Records

The scale of retrieved data made full manual screening infeasible at early stages.

Mitigation:

Early-stage automated filtering was applied to reduce the dataset size prior to detailed review, enabling subsequent manual analysis to focus on a manageable subset of repositories.

6. Manual Validation and Repository Screening

Following automated filtering, a manual validation phase was conducted to assess repository relevance and quality.

Each shortlisted repository was individually reviewed by:

- Opening the repository URL
- Reading and analysing the README file
- Assessing project purpose, scope, and technical content

- Evaluating alignment with AI development, AI security, open-source governance, or risk-relevant use cases

Repositories were retained only if they demonstrated clear relevance to the research objectives. Projects that were tangential, deprecated, purely educational without analytical value, or unrelated to AI were excluded.

This manual review ensured that the final dataset reflected substantive grey literature, rather than superficial keyword matches.

7. Reproducibility and Transparency

All extraction, cleaning, and filtering steps were implemented using reproducible Python scripts. Query logic, rate-limit handling, and filtering thresholds were explicitly defined, allowing the process to be rerun or audited if required.

The methodology prioritised transparency, traceability, and methodological defensibility, in line with DSIT research standards.

8. Summary

This GitHub API data extraction process produced a large-scale, systematically collected grey-literature dataset capturing the intersection of open-source software and artificial intelligence. By combining automated API collection with structured filtering and manual validation, the methodology strikes a balance between scale and relevance, as well as analytical depth.

The resulting dataset provides a robust empirical foundation for identifying emerging AI-related risks, development patterns, and governance gaps within open-source ecosystems.